HW2 — Motion Estimation (VC)

https://hackmd.io/@G2sNDYsCS7y7FXp11rLqBQ/BkvnqHJhMl

(https://hackmd.io/@G2sNDYsCS7y7FXp11rLqBQ/BkvnqHJhMl)

> This assignment extends the HW1 Spatial Coding codec. Students must implement **Motion Estimation (ME), Motion Compensation (MC), an Encoder, and a Decoder**. Q3 additionally requires **Rate-Distortion Optimization (RDO)** to select the motion block size. All three questions share the HW1 8×8 residual codec and the fixed bitstream format. They differ only in the ME method and block-size decision.

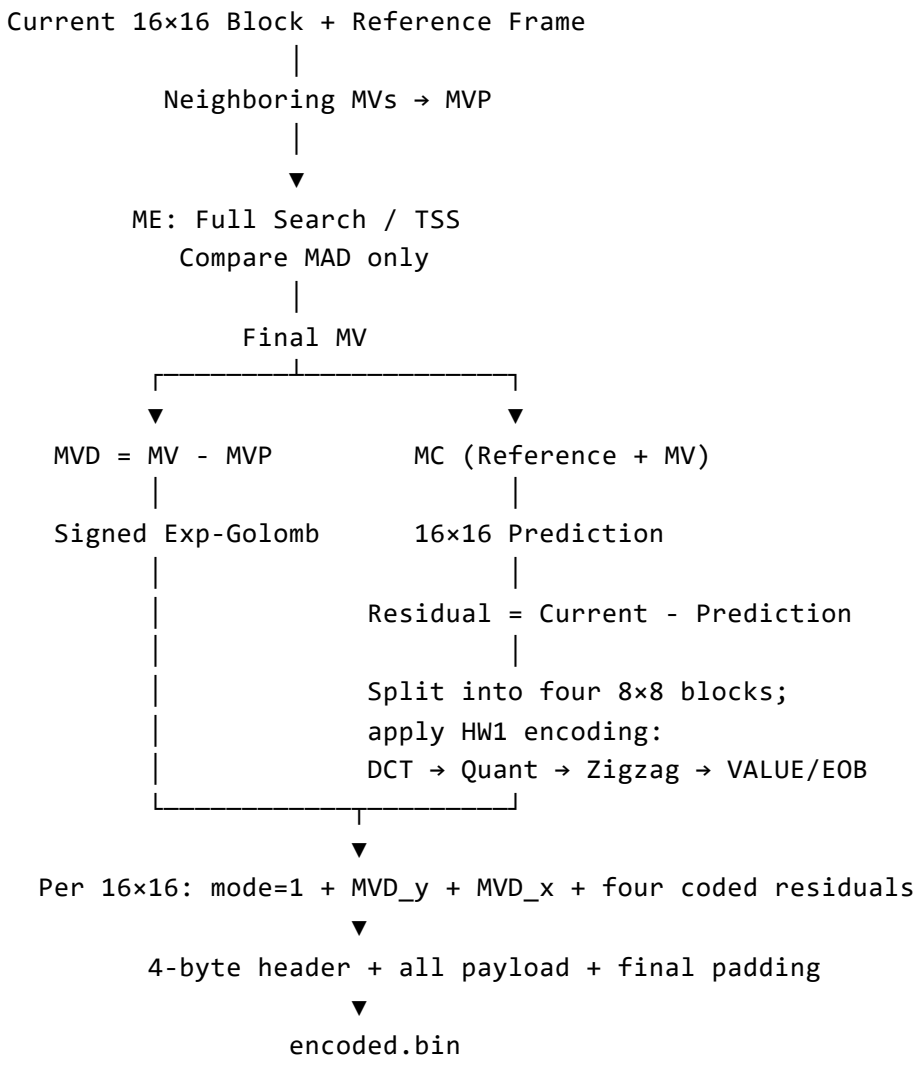
1. Assignment Objectives and Common Specifications

- `one_gray.png` : **Reference Frame**, known to both the Encoder and Decoder. Its file size is **not included in the BPP** for the current frame.
- `two_gray.png` : **Original Current Frame**, accessible to the Encoder only; used to calculate the final PSNR.
- Both images must have the same dimensions, be **8-bit grayscale**, and have widths and heights divisible by **16**.
- Process the image one **16×16 macroblock** at a time, from left to right and top to bottom (**raster order**). Define the Motion Vector as `MV=[dy,dx]`, with positive `dy` pointing downward and positive `dx` pointing rightward. Search displacements at integer-pixel resolution.
- Each candidate block must lie entirely within the Reference Frame: **no padding and no negative-index wraparound**.
- **Motion prediction block size and transform block size are different.** Even a 16×16 prediction produces four **8×8 residual transform blocks**, using the HW1 8×8 DCT / Quantization / Zigzag / VALUE-EOB codec.

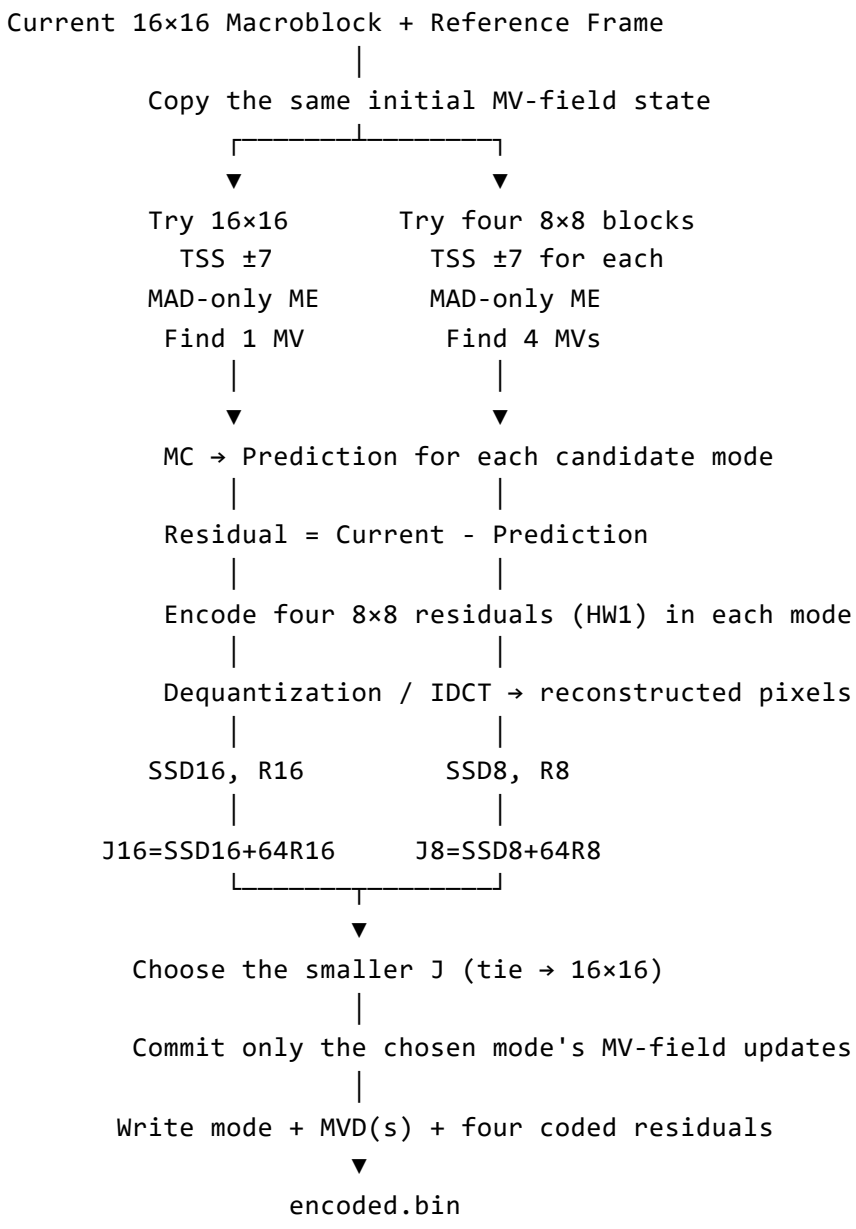
Question	ME Search	Cost Used to Select MV	Motion Block Size	RDO Required?
Q1 Full Search	±7	**MAD only**	Fixed 16×16 (1 MV)	No
Q2 Three-Step Search (TSS)	±7, `4→2→1`	**MAD only**	Fixed 16×16 (1 MV)	No
Q3 Three-Step Search + RDO	±7, `4→2→1`	**MAD only**	One 16×16 (1 MV) or four 8×8 blocks (4 MVs)	**Yes**

> **Key points:**
>
> - **All three questions use MAD only for MV selection during ME.**
> - **MVD must still be encoded using Signed Exp-Golomb and stored in the bitstream.** Only Q3 additionally includes the MVD bit cost in `R_bits` when making the block-size RDO decision.

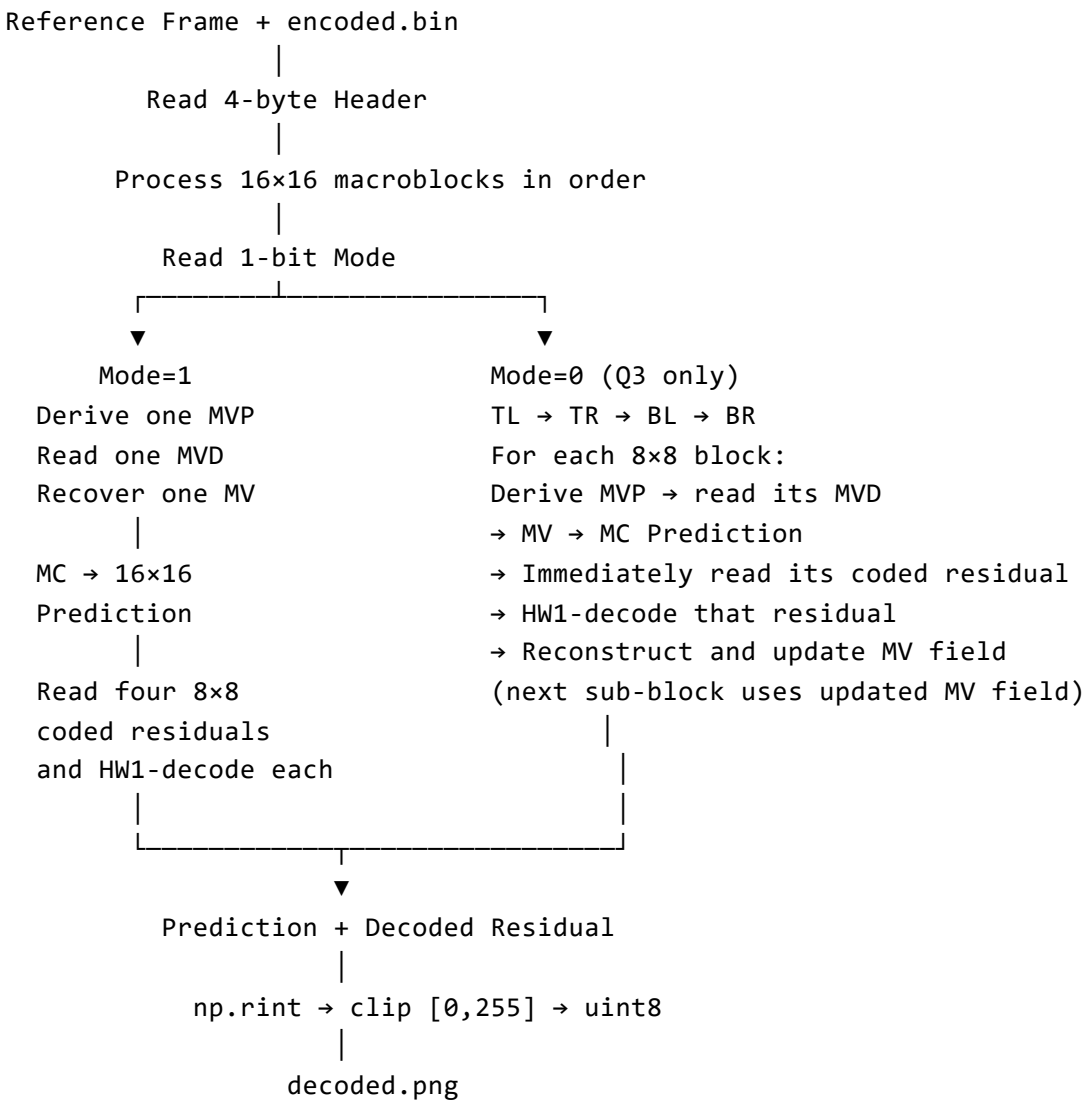
1.1 Q1/Q2 Encoder: Fixed 16×16, MAD-only ME



1.2 Q3 Encoder: MAD-only TSS + Block-size RDO



1.3 Shared Decoder (Q1/Q2/Q3)



Actual bitstream reading order matters: for `mode=1`, read **one MVD pair**, followed by **four residual blocks**. For `mode=0`, process TL→TR→BL→BR; **each 8×8 sub-block stores its MVD followed immediately by its residual**, rather than storing all four MVDs before all four residuals. Residual decoding for both modes uses the HW1 pipeline: `VALUE/EOB → Inverse Zigzag → Dequantization → IDCT`.

The Decoder does not perform ME searching or RDO again. The Encoder has already recorded the mode and MVDs in `.bin`; the Decoder can reconstruct the image using the same MVP algorithm, MC, and the inverse HW1 codec.

2. Motion Estimation (ME)

For block size `B=16` (Q1/Q2 and Q3 large mode) or `B=8` (Q3 small mode), compute for every valid candidate:

$$MAD(MV) = \frac{1}{B^2} \sum_{i,j=0}^{B-1} |Current(i,j) - Candidate(i,j)|.$$

Convert pixel values to a signed integer or floating-point type **before subtraction** to prevent `uint8` overflow.

Q1, Q2, and Q3 all select motion vectors using MAD only:

$$MV^* = \arg\min_{MV} MAD(MV).$$

If multiple candidates have **exactly the same MAD**, use the following deterministic tie-breaking order: smaller `|dy|+|dx|`, then smaller `dy`, then smaller `dx`. **Do not include MVP/MVD bit costs in ME searching.** After finding the final MV, the Encoder derives its MVD from the fixed MVP and writes it using Signed Exp-Golomb. In Q3, the corresponding MVD bits are also included in the RDO Rate calculation in Section 4.

Full Search: search all legal integer-pixel displacements with `dy,dx ∈ [-7,7]`, for up to $15 \times 15 = 225$ candidate positions. **TSS:** search within `±7`, starting at `center=[0,0]`, with three consecutive 3×3 search stages using step sizes **`4→2→1`**. At each stage, update the center to the valid candidate with the lowest MAD. **All three stages must be completed; no early termination is allowed.** All methods use the same boundary constraints.

3. Motion Vector Coding: MVP, MVD, and Signed Exp-Golomb

3.1 Why Are MVP and MVD Needed?

The Encoder and Decoder each maintain an MV field indexed by **8×8 cells**, initially set to `[0,0]` everywhere. Use the supplied `provided/predictor_mvd.py`: compute the component-wise median of `dy` and `dx` from already completed **Left, Top, and Top-Right** neighboring MVs. Missing or not-yet-completed neighbors are treated as `[0,0]`.

$$MVD = MV - MVP, \qquad MV = MVP + MVD.$$

Both sides can derive the **same MVP** from completed neighboring blocks. Therefore, only MVD is transmitted; MVP itself is not stored.

For example, if `MV=[6,1]` and `MVP=[5,1]`, then `MVD=[+1,0]`. MVD values are usually smaller and can often be encoded with shorter variable-length codewords.

3.2 How Does Signed Exp-Golomb Encoding Work?

First map a signed integer v to a nonnegative integer:

$$codeNum(v) = \begin{cases} 2v - 1, & v > 0 \\ -2v, & v \le 0 \end{cases}.$$

Let $n = codeNum(v) + 1$ and $k = \lfloor \log_2 n \rfloor$. To form the actual **binary codeword**, write k leading `0` bits followed by the `(k+1)`-bit binary representation of n.

$$Bits(v) = 2\lfloor \log_2(codeNum(v) + 1) \rfloor + 1.$$

Original v	$codeNum$	Exp-Golomb Binary	Bits
0	0	`1`	1
+1	1	`010`	3
-1	2	`011`	3
+2	3	`00100`	5
-2	4	`00101`	5
+3	5	`00110`	5
-3	6	`00111`	5

Encoding example: `MVD_y=+1 → codeNum=1 → n=2 → binary 10 → prefix one 0 → 010`; `MVD_x=0 → 1`. Concatenate the two components as **`010 | 1` (4 bits)**, with no additional field delimiter.

3.3 How Does the Decoder Recover Variable-length MVDs?

There is **no need to store the bit length of each MVD**, because Exp-Golomb is a **self-delimiting prefix code**:

1. At the current bit position, count consecutive leading `0`s. If there are k zeros, read **exactly k more bits** after the first `1`.

2. The first `1` plus those k following bits form the `(k+1)` -bit binary representation of n. Recover `codeNum = n - 1` .
3. If `codeNum` is odd, recover `v=(codeNum+1)/2` ; if it is even, recover `v=-codeNum/2` .
4. The BitReader now points to the next field, so decoding continues with the next MVD component or the coded residual.

For `010 | 1` , the first field begins with one leading zero. Reading `010` gives `n=2` , `codeNum=1` , and `v=+1` . The second field begins with `1` (zero leading zeros), giving `v=0` .

3.4 From Mode + MVD + Residual to `encoded.bin`

Suppose **one 16×16 macroblock** selects `mode=1` and has `MVD=[+1,0]` . For this illustration, assume all coefficients of its four quantized 8×8 residual blocks are zero, so each block is represented by only the HW1 `EOB=11111111` (this is an illustrative example, not the actual image output):

```
Mode                1                   1 bit
MVD_y (+1)          010                 3 bits
MVD_x (0)           1                   1 bit
Residual TL         111111111           9 bits
Residual TR         111111111           9 bits
Residual BL         111111111           9 bits
Residual BR         111111111           9 bits

Payload (no actual field separators) = 1|010|1|EOB|EOB|EOB|EOB
valid_payload_bits = 41
Padding = seven 0 bits (only at the end of the complete payload)
4-byte header (big-endian) = 00 00 00 29
Complete example encoded.bin (hex) = 00 00 00 29 AF FF FF FF FF 80
```

The `|` characters are **for readability only; the actual bitstream contains no `|` characters**. The header stores **the number of valid bits in the entire payload**, not the lengths of individual fields. A HW1 residual block ends when its EOB is encountered or when all 64 coefficients have been read. The Decoder follows the fixed field order. In a real `.bin` file, the macroblocks are concatenated consecutively, and padding is added only once at the very end.

Provided example code for students: `provided/exp_golomb_demo.py` includes `encode_signed()` and `pack_payload()` and uses the student-visible evaluator's `BitReader` to decode the above example `00 00 00 29 AF FF FF FF FF 80` . Run it from the student package root:

```
python provided/exp_golomb_demo.py
```

This example demonstrates **bit-level writing and reading only**; it does not provide the ME, RDO, or complete Encoder/Decoder solutions.

3.5 Special Considerations for MVP/MVD When the Block Size Changes

Q1/Q2 and Q3's 16×16 mode: derive **one MVP** from the current macroblock's **top-left 8×8 cell**, and encode only **one MVD pair** for the entire 16×16 macroblock. Once the mode is selected, fill the MV field's **2×2 cells** with that same MV so later blocks can use it for prediction.

Q3's four-8×8 mode: process sub-blocks in the order **TL (top-left) → TR (top-right) → BL (bottom-left) → BR (bottom-right)**. At each step, use the **currently updated MV field** to derive the MVP, write that sub-block's MVD, and then update its MV-field cell. All four MVD pairs must be encoded.

For example, suppose all four 8×8 sub-blocks in a macroblock select `MV=[1,0]`, and previously completed neighboring MVs outside this macroblock are all `[0,0]`. With the provided median predictor:

Order	MVP	MV	MVD	Explanation
TL	`[0,0]`	`[1,0]`	`[1,0]`	First sub-block; surrounding MVs are zero.
TR	`[0,0]`	`[1,0]`	`[1,0]`	Left is 1, while Top/Top-Right are 0; median is still 0.
BL	`[1,0]`	`[1,0]`	`[0,0]`	Top and Top-Right now contain 1.
BR	`[1,0]`	`[1,0]`	`[0,0]`	Left and Top now contain 1.

Even if the four **MVs are identical, their MVDs may differ**. An `MVD=[0,0]` still requires encoding `1|1` (2 bits). Do not omit repeated values, because doing so would misalign the Decoder's bitstream parsing.

RDO state isolation: when evaluating the 16×16 and four-8×8 modes, start each trial from the **same MV-field state after the previous macroblock has been finalized**. The unselected mode must not alter the actual MV field. Otherwise the two candidates may use different MVP states, producing incorrect R values and breaking Encoder/Decoder synchronization.

4. Q3 Only: Rate-Distortion Optimization (RDO)

All three questions use MAD-only ME to find MVs; only Q3 additionally performs block-size RDO. For the same 16×16 macroblock, evaluate:

- **Large:** 16×16 prediction, one MV/MVD pair, and four 8×8 residual blocks.
- **Small:** four 8×8 predictions, four MV/MVD pairs, and four 8×8 residual blocks.

For **both modes**, perform MC, DCT, quantization, residual coding, inverse operations, and reconstruction before computing the distortion from the actual **Reconstructed** pixels. **Do not use MAD as the RDO distortion D.**

4.1 Common Definitions of D, R, and λ

This assignment uses the following fixed formulation:

$$\boxed{J_{RDO} = SSD + 64R_{bits}}$$

$$SSD = \sum_{i,j=0}^{15} (Current_{ij} - Reconstructed_{ij})^2, \qquad R_{bits} = 1 + Bits(\text{all MVDs}) + Bits(\text{four coded resid}$$

- **D = SSD (Sum of Squared Differences):** the **sum** of squared reconstruction errors over all 256 pixels. `Reconstructed` is evaluated after `np.rint → clip[0,255] → uint8`.
- **R = actual valid syntax bits for the current macroblock:** the 1-bit mode, either one or four Signed Exp-Golomb MVD pairs, and four HW1-coded residual blocks. Do not include the shared 4-byte file header or final padding. **Do not use the whole-image BPP for this local RDO decision.**
- **λ = 64:** the assignment's fixed weight. Students may not tune it to improve scores. If the two J values are identical, select `mode=1` (16×16).

Background references: H.264/AVC mode decision and λ (IEEE) (https://doi.org/10.1109/ISSPIT.2007.4458143), RDO bit cost includes mode/motion/residual (https://patents.justia.com/patent/20090219991). These provide context only: **the fixed rules in this**

assignment are authoritative.

4.2 RDO Calculation Example

Suppose that, after **complete encoding and reconstruction** of the same 16×16 region, the two modes yield:

Mode	SSD	R (Actual Valid Bits)	J = SSD + 64R
Large 16×16	3,072	120	10,752
Small 4×8×8	1,280	160	11,520

Choose **Large** in this example: its smaller number of bits outweighs its somewhat higher reconstruction error. If Small's SSD drops further to 600, `600 + 64×160 = 10,840` , which is still slightly above Large. If it drops to 400, `400 + 64×160 = 10,640` , then **Small** is selected instead.

RDO is a local greedy decision: compare the current macroblock's reconstruction error and coding bits without including the future cost of later macroblocks. After selecting a mode, formally write its bitstream data, update the actual MV field, and proceed to the next macroblock.

5. Fixed Bitstream Format, BPP, and PSNR

All three questions must use the same fixed format that can be parsed by the TA's canonical decoder. **Q1/Q2 must always write** `mode=1` ; only Q3 may choose between `mode=1` and `mode=0` . For a consistent format, **the 1-bit mode is still present in every macroblock**, even when Q1/Q2 use a fixed block size.

```
4-byte big-endian uint32: valid_payload_bits
    (does not include the header itself or final zero padding)

For each 16×16 macroblock in row-major order:
    1 bit: block_mode (1 = 16×16, 0 = four 8×8 blocks)

    if mode == 1:
        Signed Exp-Golomb(MVD_y)     # One 16×16 MV
        Signed Exp-Golomb(MVD_x)
        HW1 VALUE/EOB(coded residual TL 8×8)
        HW1 VALUE/EOB(coded residual TR 8×8)
        HW1 VALUE/EOB(coded residual BL 8×8)
        HW1 VALUE/EOB(coded residual BR 8×8)

    else:  # Q3 only
        for 8×8 sub-block in TL → TR → BL → BR:
            Signed Exp-Golomb(MVD_y)
            Signed Exp-Golomb(MVD_x)
            HW1 VALUE/EOB(coded residual 8×8)

0–7 zero-padding bits (only at the end of the entire payload)
```

MVD/residual ordering depends on the Mode and must match exactly between Encoder and Decoder. Do not store separate MVD codeword lengths, MVP values, or the macroblock count, because the fixed format and image dimensions determine the parsing structure. Do not add custom magic bytes, extra fields, or per-block padding.

$$BPP = \frac{8 \times FileSize(encoded.bin \text{ in bytes})}{Width \times Height}, \qquad PSNR = 10\log_{10}\frac{255^2}{MSE(two_gray, decoded)}.$$

BPP counts all bytes in the actual `.bin` file, including header, mode, MVD, residual, and padding. **PSNR** compares the original `two_gray.png` with `decoded.png` obtained by truly decoding `.bin`. The Decoder **must not read** `two_gray.png`.

6. Student Implementation, Provided Files, and Submission

Students must implement `src/encoder.py`, `src/decoder.py`, ME, MC, Q3 RDO, bit-level reading/writing, and integration with the HW1 codec. You may add Python helper files, but must not modify the fixed DCT/Quantization/VALUE-EOB coding, MVP algorithm, or bitstream format.

Provided Reference and Validation Tools

File	Purpose
`hw1_reference/encoder.py`, `decoder.py`, `evaluator.py`	Reference implementation for HW1 spatial coding
`provided/predictor_mvd.py`	**Fixed MVP/MVD** helper; do not modify its algorithm
`provided/exp_golomb_demo.py`	**Signed Exp-Golomb example code:** binary writing → header/padding → read MVD back
`provided/evaluator.py`	Check Mode, MVD, HW1 residual, MV range, Header/Padding, and complete payload syntax; compute PSNR from submitted PNG and BPP from `.bin`
`provided/check_submission.py`	Check the unpacked submission structure and bitstream syntax for all three methods
`run_local.py`	Generate submission outputs for all three methods using the fixed CLI

A student-side `SYNTAX PASS` **only confirms that the bitstream can be parsed**. It does not guarantee that ME/RDO is correct or that the submitted PNG matches the canonical reconstruction from `.bin`. The TA's independent decoder will perform that validation during official grading.

Required CLI (Used by the Grader)

```
python src/encoder.py --reference images/one_gray.png --current images/two_gray.png --metho
python src/decoder.py --reference images/one_gray.png --input /tmp/encoded.bin --method ful
```

The only permitted `--method` values are `full_search`, `three_step_search`, and `three_step_rdo`. Each method must run independently. All Python dependencies must be included in `src/`. Only the Python standard library, NumPy, and Pillow may be used; obtain TA approval before using other libraries. Do not hard-code answer `.bin` / `.png` files or replace the required search algorithm with a built-in Motion Estimation, Optical Flow, or Template Matching API.

ZIP Submission Format

```
HW2_[student_id].zip
└── HW2_[student_id]/
    ├── src/
    │   ├── encoder.py          # required
    │   ├── decoder.py          # required
    │   └── *.py                # optional student helper files
    ├── report/
    │   └── HW2_[student_id].pdf
    └── results/
        └── submitted/          # outputs generated by the student; required
            ├── full_search/
            │   ├── encoded.bin
            │   └── decoded.png
            ├── three_step_search/
            │   ├── encoded.bin
            │   └── decoded.png
            └── three_step_rdo/
                ├── encoded.bin
                └── decoded.png
```

After receiving the ZIP, the TA will safely extract it and copy the student's `src/` into a **separate execution directory**, then rerun the Encoder and Decoder to regenerate bin/png outputs. The submitted `results/submitted/` files **will not be overwritten**. The original and replayed outputs will be compared, and substantial discrepancies will be flagged for manual review. **Official PSNR/BPP scores use valid TA-replayed outputs.** The Decoder must reconstruct using only the Reference Frame and bitstream.

7. Report (14 Points)

The report is **at most 2 pages**. We recommend a single table comparing Q1/Q2/Q3 PSNR and BPP (optionally Runtime), along with concise discussion of:

1. The main search differences between Q1 Full Search and Q2 TSS (both use MAD-only ME), fixed 16×16 prediction, and MVD coding.
2. How Q3 uses the same MAD-only TSS to obtain MVs, then selects between 16×16 and four 8×8 blocks with `SSD+64R`. Briefly explain how the Rate accounts for mode/MVD/residual and report the number or proportion of macroblocks selected in each mode.
3. The image-quality versus bitrate trade-offs among the three questions. Runtime/Search Points may be included as additional analysis but do not affect automatic grading.

Do not repeatedly describe the same HW1 codec or bitstream format for all three questions. **All three use MAD-only ME**, Q1/Q2 perform no block-size selection, and **only Q3 uses RDO to choose a mode**.

8. Grading (100 Points)

Question	Valid Execution Points	PSNR / BPP	Subtotal
Q1 Full Search (16×16, ±7)	**25**	Distance from class mean: 10 / 5 / 0	**35**
Q2 TSS (16×16, ±7)	**25**	Distance from class mean: 10 / 5 / 0	**35**
Q3 TSS + RDO (±7)	—	PSNR ranking 8 + BPP ranking 8	**16**
Report	—	Manually graded, 14	**14**
Total			**100**

8.1 Q1/Q2: 25 + 10/5/0

A method earns **25 execution points** only when the replay succeeds, the TA's canonical decoder can parse the resulting bitstream, and the replay-generated PNG is **pixel-wise identical** to the TA reconstruction. An invalid method receives **0 points for that entire question**.

For Q1 and Q2 independently, compute the class arithmetic means of PSNR and BPP using **valid replayed results only**. Both metrics must meet the specified conditions:

Deviation from That Question's Class Means	Additional Points
$\lvert\mathrm{PSNR} - \mathrm{Mean\ PSNR}\rvert \le 1\,\mathrm{dB}$ **and** $\dfrac{\lvert\mathrm{BPP} - \mathrm{Mean\ BPP}\rvert}{\mathrm{Mean\ BPP}} \le 10\%$	**10**
Fails the above but **both** deviations are within `≤ 2 dB` and `≤ 20%`	**5**
Either metric exceeds the outer limits	**0**

8.2 Q3: Separate PSNR and BPP Rankings, 8 Points Each

Only Q3 submissions with a **valid replay** enter the rankings. Higher PSNR is better, and lower BPP is better. The two metrics are ranked separately; ties receive the same best rank:

Class Tier	PSNR (8 points)	BPP (8 points)
Top 20%	8	8
20–40%	6	6
40–60%	4	4
60–80%	2	2
Bottom 20%	0	0

8.3 Report, Submission Format, and Penalties

- `report/HW2_[student_id].pdf` must be at most 2 pages and is graded manually out of **14 points**; a missing report receives 0.
- `src/encoder.py` , `src/decoder.py` , and each question's `results/submitted/encoded.bin` and `decoded.png` must exist. If a question cannot be replayed, has an invalid bitstream, or

produces a PNG inconsistent with the canonical reconstruction, that question receives no points.

- If the TA must **manually correct an incorrect filename** before grading, apply the existing rules for invalid components and record `manual_filename_fix=true` in the CSV after the correction. The **final grade is multiplied by 0.9**. The grader must not silently repair filenames.
- A failure by one student or in one method does not stop the rest of the batch. The TA Grader stores separate replay logs, warnings for discrepancies with submitted outputs, and checkpoint/resume support using the same run ID.

> **Before submitting, run `provided/check_submission.py .`** A Student Evaluator PASS confirms only the folder structure and bitstream syntax; it does not replace the TA's canonical decoder or replay testing.